

D.4.3 — Project Full Application

Vector Index Benchmarking Framework

Course: COMP-8157 — Advanced Database Topics

Project: Performance Evaluation of Vector Indexing Techniques for AI Search Systems

Group: Group 4

Deliverable: D.4.3 — Project Full Application

1. Project Overview

The **Vector Index Benchmarking Framework (VecBench)** is a Python-based experimental benchmarking application developed to evaluate and compare vector indexing techniques used in artificial intelligence and semantic search systems.

Modern AI search applications represent text and other data as high-dimensional vectors. Searching these vectors efficiently becomes increasingly important as dataset size grows. Different vector indexing techniques provide different trade-offs between search accuracy, latency, memory consumption, index construction time, and throughput.

The project evaluates three major vector indexing approaches:

- **Flat Index** — performs exhaustive exact nearest-neighbour search and serves as the ground-truth baseline.
- **IVF (Inverted File Index)** — partitions vectors into clusters and searches selected partitions to improve performance.
- **HNSW (Hierarchical Navigable Small World)** — uses a graph-based approximate nearest-neighbour search structure.

All indexing techniques are evaluated under controlled experimental conditions using the same datasets, vectors, query workloads, and evaluation procedures.

The major performance metrics evaluated by the framework are:

- Query latency
- P50 latency
- P95 latency
- Recall@K
- Queries per second (QPS)
- Index memory/serialized size
- Index construction time
- Scalability with increasing dataset size

The application supports both real semantic-search workloads using text embeddings and synthetic vector workloads for controlled scalability experiments.

2. Source Code Repository and Project Portal

Source Code Repository:

[<https://github.com/Devz0201/COMP8157-Vector-Indexing>]

The GitHub repository serves as the primary project portal for the D.4.3 submission.

The repository contains the complete application, including:

- Python source code
- Benchmarking modules
- Vector indexing implementations
- Query and evaluation components
- Dataset files
- Dataset preparation utilities
- YAML experiment configurations
- SQLite database and schema
- Automated tests
- GitHub Actions continuous integration configuration
- Generated CSV benchmark results
- Generated visualizations
- Google Colab notebook
- Project documentation
- Project-management documentation
- Deployment documentation
- User documentation
- Team contribution documentation

The repository is organized as a complete development project and provides the files required to reproduce, execute, test, and evaluate the application.

3. Project Management Tool

Project Management Platform: Hive

Project Board:

<https://app.hive.com/workspace/JbYCPPC6L8DCGyzKG?actionViewId=W4yJq7vXwthAk2DqF&tabId=aprxRosGk2vxPJixC&projectId=48q28Z9gpJfRKWDJs>

Hive was used to support the planning, coordination, implementation, and tracking of project activities.

The project-management process covers task organization, work assignment, development progress, project milestones, completed work, documentation activities, testing activities, and deliverable preparation.

Project-management evidence is also documented in:

docs/PROJECT_MANAGEMENT.md

The project implementation demonstrates requirements management, scope organization, work breakdown, iterative development, version control, testing, configuration management, documentation, risk management, quality assurance, and team collaboration.

4. Technology Stack

Component	Technology
Programming Language	Python
Vector Search	FAISS
Numerical Processing	NumPy
Data Processing	Pandas
Visualization	Matplotlib
Machine Learning/Data Utilities	scikit-learn
Configuration	YAML / PyYAML
Semantic Embeddings	Sentence-Transformers
Embedding Model	all-MiniLM-L6-v2
Database	SQLite 3
Testing	pytest
Code Quality	Ruff
Continuous Integration	GitHub Actions

Interactive Cloud Execution

Google Colab

Cloud Development Environment

GitHub Codespaces

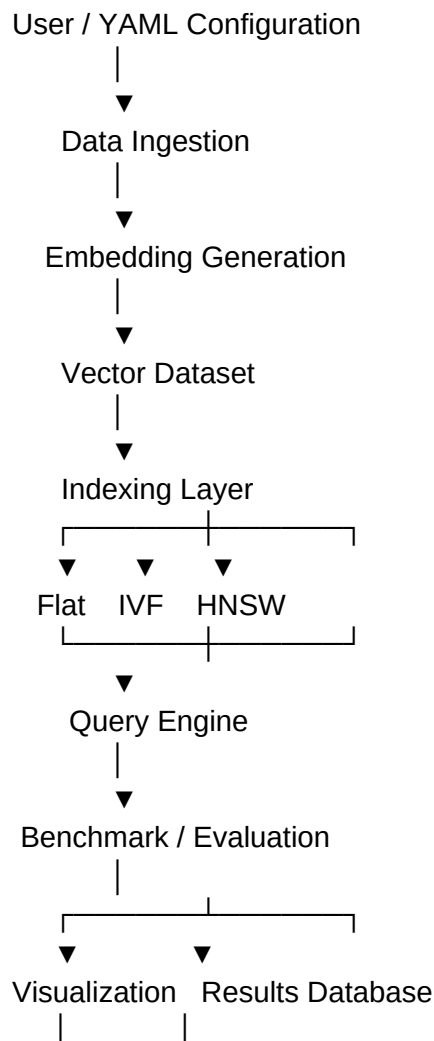
Python Packaging

pyproject.toml / requirements.txt

The application is designed primarily for CPU-based execution. Alternative embedding approaches are available for experiments that do not require downloading the Sentence-BERT model.

5. System Architecture

The system follows a modular experimental architecture in which data preparation, embedding generation, indexing, querying, evaluation, visualization, and database persistence are separated into independent components.



▼ ▼
CSV / PNG SQLite

The data-ingestion component loads either text datasets or synthetic data. Text data is converted into dense vector representations through the configured embedding backend.

The indexing layer provides Flat, IVF, and HNSW implementations through a common interface. The query engine executes the same query workload against each indexing method.

Flat search performs exhaustive exact nearest-neighbour search and provides the ground truth used to calculate Recall@K for approximate indexing methods.

The benchmark layer records latency, recall, throughput, build time, and index size. The visualization module generates CSV result files and graphical outputs, while the database component stores experiment runs, measurements, tuning points, and execution logs.

6. Application Workflow

1. The user selects an experiment configuration stored in a YAML file.
 2. The benchmark application reads the selected configuration and initializes the required experiment components.
 3. The data-ingestion component loads a text dataset or generates synthetic vector data depending on the selected experiment.
 4. For text-based experiments, the configured embedding backend converts the text into dense vector representations.
 5. The application prepares the database vectors and held-out query vectors.
 6. A Flat index is constructed to perform exact nearest-neighbour search and establish the ground-truth results.
 7. Flat, IVF, and HNSW vector indexes are constructed using the same vector data.
 8. Warm-up queries are executed before performance measurements to reduce one-time initialization and cache effects.
 9. The query workload is executed against each indexing technique.
 10. Query latency is recorded and used to calculate performance statistics including mean latency, P50 latency, P95 latency, and queries per second.
 11. Approximate-search results from IVF and HNSW are compared with Flat ground truth to calculate Recall@K.
 12. Index construction time and serialized index size are recorded.
 13. Benchmark measurements are organized and exported as CSV result files.
 14. Matplotlib generates visualizations of the benchmark results.
 15. Experiment information, configurations, environment information, measurements, tuning results, and execution logs are persisted in the SQLite results database.
-

7. Project Directory Structure

vecbench/

- README.md
- LICENSE
- requirements.txt
- pyproject.toml
- run_benchmark.py
- tune_parameters.py
- db_cli.py
- fetch_dataset.py
- prepare_headlines.py
- run_all.sh
- run_all.bat
- src/
 - __init__.py
 - benchmark.py
 - data_ingestion.py
 - embeddings.py
 - indexing.py
 - query_engine.py
 - results_db.py
 - visualization.py
- configs/
 - scalability.yaml
 - text_sbert.yaml
 - tuning.yaml
 - ci_smoke.yaml
- data/
 - headlines.txt
 - sample_corpus.txt
 - DATASET.md
- db/
 - schema.sql
 - vecbench_results.db
- tests/
 - test_indexing.py
 - test_pipeline.py
 - test_results_db.py
- results/
 - scalability/
 - sbert/
 - tuning/

```
├── notebooks/
│   └── VecBench_Colab.ipynb
├── docs/
│   ├── PROJECT_MANAGEMENT.md
│   ├── DEPLOYMENT.md
│   ├── USER_GUIDE.md
│   ├── figures/
│   └── contributions/
├── .github/
│   ├── workflows/
│   │   └── ci.yml
└── .devcontainer/
    └── devcontainer.json
```

8. Source Code File Inventory

Source File	Purpose
<code>run_benchmark.py</code>	Primary command-line entry point for benchmark experiments
<code>tune_parameters.py</code>	Performs IVF and HNSW parameter-tuning experiments
<code>db_cli.py</code>	Provides command-line access to stored benchmark results
<code>fetch_dataset.py</code>	Supports acquisition of the external headline dataset
<code>prepare_headlines.py</code>	Prepares, samples, and processes the headline dataset
<code>src/</code> <code>data_ingestion.py</code>	Handles dataset loading, cleaning, and preparation
<code>src/embeddings.py</code>	Generates vector representations using configured embedding backends
<code>src/indexing.py</code>	Implements Flat, IVF, and HNSW vector indexing
<code>src/query_engine.py</code>	Executes queries and calculates evaluation metrics

<code>src/benchmark.py</code>	Coordinates benchmark experiments and scalability testing
<code>src/results_db.py</code>	Handles persistent storage of experiment results in SQLite
<code>src/ visualization.py</code>	Generates result tables, CSV files, and graphical outputs
<code>db/schema.sql</code>	Defines the SQLite results database schema
<code>tests/ test_indexing.py</code>	Tests vector-index implementations
<code>tests/ test_pipeline.py</code>	Tests the benchmark pipeline
<code>tests/ test_results_db.py</code>	Tests database persistence functionality

9. Source Code Implementation

9.1 Data Ingestion

`src/data_ingestion.py` provides the application's input-processing layer. It converts configured input sources into a consistent corpus representation that can be consumed by the remaining benchmark components.

The module supports both text workloads and generated vector workloads. Separating data ingestion from indexing allows different datasets to be evaluated without modifying the vector-index implementation.

9.2 Embedding Generation

`src/embeddings.py` provides interchangeable embedding mechanisms.

The verified embedding approaches are:

- Sentence-BERT
- Hashing-based embeddings
- Synthetic vector generation

The Sentence-BERT backend supports semantic-search experiments using the `all-MiniLM-L6-v2` model.

Synthetic vectors are used for controlled scalability experiments where the primary objective is to evaluate indexing behaviour independently of external embedding-model execution.

9.3 Vector Indexing

`src/indexing.py` implements the primary vector indexing techniques evaluated by the project.

Major classes include:

BaseIndex
FlatIndex
IVFIndex
HNSWIndex
BuildStats

The module also provides a `build_index()` factory for creating the required indexing implementation.

Flat Index

Flat search performs exhaustive exact nearest-neighbour comparison against the vector dataset. It provides maximum retrieval accuracy and serves as the ground-truth baseline for Recall@K evaluation.

IVF Index

The IVF implementation uses FAISS `IndexIVFFlat`. Vectors are assigned to partitions, and only selected partitions are searched during query execution.

The `nprobe` parameter controls the number of partitions searched and therefore influences the trade-off between search latency and recall.

HNSW Index

The HNSW implementation uses FAISS `IndexHNSWFlat`.

HNSW organizes vectors into a navigable graph structure. The `efSearch` parameter controls the search effort and influences the relationship between query latency and retrieval quality.

The indexes use inner-product search with normalized vectors, providing cosine-style similarity behaviour.

9.4 Query Processing and Evaluation

`src/query_engine.py` handles controlled query execution and evaluation.

Major components include:

```
QueryResult
MethodMetrics
run_queries()
recall_at_k()
evaluate()
```

The evaluation process measures:

- Mean query latency
- P50 latency
- P95 latency
- Queries per second
- Recall@K
- Index size
- Index build time

Warm-up operations are performed before timed measurements to reduce the effect of one-time initialization.

9.5 Benchmark Orchestration

`src/benchmark.py` coordinates the experimental pipeline.

Important functions include:

```
run_single_scale()
run_scalability()
```

The benchmark layer ensures that the indexing techniques are evaluated against consistent vector data and query workloads, allowing meaningful comparisons between the methods.

9.6 Results Database

`src/results_db.py` provides persistent experiment storage using SQLite.

Benchmark results are therefore available both as generated CSV/visual outputs and as structured database records that can be queried across different experimental runs.

9.7 Visualization

`src/visualization.py` converts benchmark measurements into tables and graphical outputs.

Generated artifacts are stored under the `results/` directory and provide visual evidence of latency, recall, scalability, and tuning behaviour.

10. Extended Code Comments

The project's major source-code modules contain module documentation, class documentation, function documentation, algorithm explanations, parameter descriptions, and comments describing non-obvious implementation decisions.

The documentation focuses on important implementation concepts rather than commenting on obvious individual Python statements.

Major areas documented within the source include:

- Module responsibilities
- Vector-index construction
- Query execution
- Recall calculation
- IVF configuration
- HNSW configuration
- Benchmark procedures
- Data processing
- Database persistence
- Error handling
- Experiment reproducibility

The SQL database schema also contains comments describing tables, relationships, constraints, indexes, and views.

11. Dataset Documentation

A Million News Headlines

Attribute	Details
Dataset	A Million News Headlines

Project File	<code>data/headlines.txt</code>
Format	Plain text derived from CSV data
Original Dataset Size	Approximately 1.24 million records
Project Sample	20,000 headlines
Purpose	Real semantic-search benchmark
Original Fields	<code>publish_date, headline_text</code>
Source	Harvard Dataverse
Persistent Identifier	DOI: 10.7910/DVN/SYBGZL
Licence	CC0 1.0

The real-world dataset is used to evaluate vector indexing under a semantic text-search workload.

The project provides reproducible dataset acquisition and preparation utilities.

Dataset acquisition:

```
python fetch_dataset.py --n 20000
```

Dataset preparation from the original CSV:

```
python prepare_headlines.py \  
  --csv abcnews-date-text.csv \  
  --out data/headlines.txt \  
  --n 20000 \  
  --seed 42
```

The project also contains `data/DATASET.md`, which documents dataset provenance and preparation information.

Synthetic Dataset

Synthetic vectors are generated programmatically for scalability experiments.

The synthetic workload allows Flat, IVF, and HNSW performance to be evaluated as vector count increases without introducing the computational cost or external dependency associated with semantic embedding generation.

Because these vectors are generated internally by the application, there is no external dataset source.

Sample Corpus

`data/sample_corpus.txt` provides a small text corpus for demonstrations and lightweight execution scenarios.

12. Database Schema and Connection

Database Technology

SQLite 3

Database File

`db/vecbench_results.db`

SQLite was selected because it provides persistent structured experiment storage without requiring a separate database server.

No hostname, database server, username, or password is required.

Database Schema

Table	Primary Key	Purpose
<code>run</code>	<code>run_id</code>	Stores benchmark invocation and environment information
<code>measurement</code>	<code>measurement_id</code>	Stores performance measurements for indexing methods
<code>tuning_point</code>	<code>tuning_id</code>	Stores IVF/HNSW parameter-tuning measurements
<code>run_log</code>	<code>log_id</code>	Stores benchmark execution logs

The database follows the general relationship:

```
run
├── measurement
├── tuning_point
└── run_log
```

Child records are associated with their corresponding benchmark run through `run_id`.

The schema contains validation constraints for important experimental values, including valid recall ranges and controlled experiment/method values.

Database views include:

```
v_results
v_method_summary
v_scalability
v_tuning_curve
```

These views simplify analysis of benchmark results, scalability experiments, method comparisons, and tuning curves.

Database Connection

Database initialization:

```
python db_cli.py init
```

List experiment runs:

```
python db_cli.py runs
```

Display summary information:

```
python db_cli.py summary
```

Display the latest run:

```
python db_cli.py show latest
```

Display benchmark results:

```
python db_cli.py results --run latest
```

Display execution logs:

```
python db_cli.py logs --run latest
```

The database can also be accessed directly using SQLite:

sqlite3 db/vecbench_results.db

No private credentials are required.

13. Dependencies

Dependency	Version Requirement Where Specified	Purpose
faiss-cpu	>=1.8.0	Flat, IVF, and HNSW vector indexing/search
numpy	>=1.24	Vector and numerical operations
pandas	>=2.0	Benchmark result processing
matplotlib	>=3.7	Result visualization
scikit-learn	>=1.3	Machine-learning/data utilities
PyYAML	>=6.0	YAML configuration processing
sentence-transformers	>=2.2 where documented	Sentence-BERT semantic embeddings
pytest	Project testing	Automated test execution
Ruff	Project quality assurance	Python linting/code-quality checking

The project's main dependency definitions are maintained through `requirements.txt` and `pyproject.toml`.

14. Installation and Configuration

Prerequisites

The primary prerequisites are:

- Python 3.10 or later
- pip
- Git

The real Sentence-BERT experiment may require internet access during the initial model download if the required model is not already cached.

Repository Setup

```
git clone [https://github.com/Devz0201/COMP8157-Vector-Indexing]
cd COMP8157-Vector-Indexing
```

Dependency Installation

```
pip install -r requirements.txt
```

Sentence-BERT support can be installed using:

```
pip install sentence-transformers
```

The application does not require private database credentials or API keys for normal execution.

Experiment parameters are controlled through the YAML files located under:

```
configs/
```

15. Running the Application

Scalability Benchmark

```
python run_benchmark.py --config configs/scalability.yaml
```

The scalability benchmark uses synthetic vector data and provides a convenient execution path without requiring an external semantic embedding model.

Sentence-BERT Benchmark

```
python run_benchmark.py --config configs/text_sbert.yaml
```

This experiment evaluates vector indexing using semantic embeddings generated from the text dataset.

Parameter Tuning

```
python tune_parameters.py --config configs/tuning.yaml
```

The tuning experiment evaluates the effect of IVF **nprobe** and HNSW **efSearch** values on the latency/recall trade-off.

Complete Workflow

Linux/macOS:

```
./run_all.sh
```

Offline execution:

```
./run_all.sh --offline
```

Windows:

```
run_all.bat
```

Results

Generated benchmark artifacts are stored primarily under:

results/

Structured experiment information is stored in:

db/vecbench_results.db

Database summaries can be displayed using:

```
python db_cli.py summary
```

16. Executable / Runnable Application

VecBench is a Python-based benchmarking application and therefore does not require a standalone native `.exe` executable for normal operation.

Runnable application entry points are provided through:

```
run_benchmark.py
tune_parameters.py
db_cli.py
run_all.sh
run_all.bat
notebooks/VecBench_Colab.ipynb
```

The Python scripts provide direct command-line execution, while `run_all.sh` and `run_all.bat` provide platform-specific workflow launchers.

The Google Colab notebook provides an additional browser-based execution mechanism.

17. Auxiliary and Supporting Files

The application contains supporting files required for reproducibility, execution, evaluation, and documentation.

Configuration Files

```
configs/*.yaml
```

These files define experiment parameters independently from application source code.

Dataset Files

```
data/headlines.txt
data/sample_corpus.txt
data/DATASET.md
```

Database Files

```
db/schema.sql
db/vecbench_results.db
```

Test Files

```
tests/test_indexing.py
```

tests/test_pipeline.py
tests/test_results_db.py

Result Files

results/**/*.csv
results/**/*.png

System Documentation

docs/

The documentation directory contains project-management, deployment, user-guide, design, requirements, final-report, contribution, and diagram materials.

Continuous Integration

.github/workflows/ci.yml

Cloud Development Configuration

.devcontainer/devcontainer.json

These supporting files allow the project to be reproduced, tested, evaluated, and understood without relying only on the source-code modules.

18. Testing and Validation

Automated testing is implemented using **pytest**.

The principal test files are:

tests/test_indexing.py
tests/test_pipeline.py
tests/test_results_db.py

test_indexing.py validates vector-index behaviour.

test_pipeline.py validates the integration and operation of the benchmark pipeline.

test_results_db.py validates SQLite persistence and database-related functionality.

Continuous integration is implemented through GitHub Actions using:

`.github/workflows/ci.yml`

The CI workflow performs code-quality checking, automated testing, and a benchmark smoke test.

The workflow executes operations including:

```
ruff check .  
pytest -v  
python run_benchmark.py --config configs/ci_smoke.yaml
```

The generated results database is subsequently accessed through the database CLI to verify that benchmark execution and persistence operate together correctly.

The project documentation records **43 passing automated tests**.

The CI configuration supports multiple Python versions and operating-system environments, improving the reproducibility of the application.

19. Project Management Implementation

The development of the Vector Index Benchmarking Framework demonstrates multiple software and project-management principles.

Requirements Management

Project requirements and expected system behaviour are formally documented through the project's D.4.1 requirements material.

Scope Management

The project scope is focused on controlled evaluation of Flat, IVF, and HNSW indexing techniques and the supporting data, query, measurement, persistence, visualization, and testing infrastructure.

Work Breakdown

Implementation responsibilities are divided into functional areas such as data/embedding processing, vector indexing, query evaluation, experimentation, visualization, testing, and documentation.

Version Control

Git is used for source-code version control, while GitHub serves as the central repository and project portal.

Project Tracking

Hive provides task and progress management for project activities.

Configuration Management

Experiment parameters are externalized into YAML configuration files rather than being embedded throughout the application source code.

This allows experiments to be modified and reproduced while preserving the underlying implementation.

Testing and Quality Assurance

Automated pytest tests validate indexing, pipeline, and database functionality.

Ruff provides automated Python code-quality checking.

GitHub Actions integrates testing and quality checks into the development workflow.

Risk Management

Project-management documentation contains identified project risks and corresponding mitigation approaches.

Change Management

Experimental findings were used to refine benchmark configuration. In particular, IVF query behaviour was evaluated through parameter tuning, allowing the `nprobe` configuration to be adjusted based on measured recall/performance behaviour.

This demonstrates an evidence-driven development process in which configuration changes are supported by benchmark results.

Documentation Management

The repository contains dedicated requirements, design, deployment, user-guide, project-management, contribution, and final-report documentation.

20. Deployment / Cloud Execution

The application supports cloud-based execution through Google Colab and GitHub Codespaces.

The project is an experimental vector-index benchmarking framework rather than a continuously running web application. Therefore, its deployment model focuses on providing reproducible execution environments instead of a permanent web frontend.

Google Colab allows the benchmark workflow to be executed through a browser-based notebook environment.

GitHub Codespaces provides a cloud development environment based on the repository configuration and allows the project to be executed without requiring the grader to manually configure a complete local development environment.

Open in Google Colab: https://colab.research.google.com/github/Devz0201/COMP8157-Vector-Indexing/blob/main/notebooks/VecBench_Colab.ipynb

Open in GitHub Codespaces: <https://codespaces.new/Devz0201/COMP8157-Vector-Indexing>

The repository also contains:

`docs/DEPLOYMENT.md`

which provides supporting deployment information.

21. Grader Quick Start

Local Execution

Clone the repository:

```
git clone [https://github.com/Devz0201/COMP8157-Vector-Indexing]
```

Open the project:

```
cd COMP8157-Vector-Indexing
```

Install dependencies:

```
pip install -r requirements.txt
```

Run the scalability benchmark:

```
python run_benchmark.py --config configs/scalability.yaml
```

View the stored result summary:

```
python db_cli.py summary
```

Generated result files are available under:

```
results/
```

and persistent experiment records are stored in:

```
db/vecbench_results.db
```

Full Reproduction

For the complete workflow with Sentence-BERT support:

```
pip install sentence-transformers  
./run_all.sh
```

On Windows:

```
run_all.bat
```

22. Application Results

The application generates experimental evidence in CSV, graphical, and database formats.

The benchmark results demonstrate the trade-offs between exact and approximate vector indexing.

Flat search provides exact nearest-neighbour results and therefore acts as the accuracy baseline.

IVF improves search efficiency by limiting search to selected vector partitions. Its performance and recall depend partly on the selected `nprobe` value.

HNSW performs approximate nearest-neighbour search using a navigable graph structure. Its search effort and retrieval quality are influenced by parameters such as `efSearch`.

The application evaluates these methods using consistent experimental workloads so that differences in latency, recall, throughput, memory, construction time, and scalability can be compared.

23. Security and Credentials

The application does not require private database credentials for normal operation.

SQLite is used as an embedded database, so no database hostname, username, password, or external database server is required.

No private password or API credential is required for the standard benchmark workflow.

Sensitive credentials should never be committed to the source-code repository. Repository access and project-management permissions are managed through the respective GitHub services rather than through credentials stored in the application.

24. Known Limitations

Sentence-BERT experiments may require a one-time model download when the required embedding model is not already available in the execution environment.

Benchmark latency measurements can vary between machines because hardware, operating-system conditions, background workloads, and available computational resources influence performance.

The synthetic scalability workload is intended primarily to evaluate vector-indexing behaviour and does not represent the computational cost of generating real semantic embeddings.

The application is designed as a command-line and notebook-based experimental framework rather than a graphical web application.

IVF and HNSW are approximate-search techniques, and their performance/accuracy trade-offs depend on configuration parameters and dataset characteristics. The project addresses this behaviour through controlled parameter-tuning experiments.

25. Project Resources

Resource	Location
Source Code Repository	[https://github.com/Devz0201/]

	<code>COMP8157-Vector-Indexing]</code>
Project Management Board	<code>[https://app.hive.com/ workspace/JbYCPPC6L8DCGyzKG? actionViewId=W4yJq7vXwthAk2DqF &tabId=aprxRosGk2vxPJixC&proje ctId=48q28Z9gpJfRKWDJs]</code>
Dataset Documentation	<code>data/DATASET.md</code>
Database Schema	<code>db/schema.sql</code>
Results Database	<code>db/vecbench_results.db</code>
Project Management Documentation	<code>docs/PROJECT_MANAGEMENT.md</code>
Deployment Documentation	<code>docs/DEPLOYMENT.md</code>
User Guide	<code>docs/USER_GUIDE.md</code>
Automated Tests	<code>tests/</code>
Benchmark Results	<code>results/</code>

26. Conclusion

The Vector Index Benchmarking Framework provides a complete and reproducible environment for evaluating Flat, IVF, and HNSW vector indexing techniques for AI and semantic-search workloads.

The submitted project contains the complete Python source code, experiment configurations, real and synthetic data support, dataset provenance documentation, SQLite database and schema, benchmark result files, visualizations, automated tests, continuous integration configuration, project-management evidence, deployment documentation, cloud execution options, and supporting project documentation.

The modular architecture separates data ingestion, embedding generation, vector indexing, query execution, benchmark evaluation, visualization, and persistent experiment storage. This structure allows the indexing techniques to be evaluated under consistent experimental conditions while keeping the application maintainable and reproducible.

The GitHub repository serves as the primary D.4.3 project portal, while Hive provides project-management evidence. Google Colab and GitHub Codespaces provide cloud-based

execution options for graders who prefer to evaluate the application without performing a complete local setup.

Together, the repository, application source code, datasets, database, testing infrastructure, generated results, documentation, and project-management materials constitute the complete